Automatic Sensitivity Labeling at Scale — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Automatic Sensitivity Labeling at Scale (Microsoft Purview) 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

Scope boundary and deferrals . . . 4

Licensing note . . . 4

The bottom line . . . 5

1. Solution Overview — Automatic Sensitivity Labeling at Scale (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 5

The two engines (and where each applies) . . . 6

Licensing (verify your licensing/tenant) . . . 6

Naming convention (used in every doc and script) . . . 6

What you can control . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Automatic Sensitivity Labeling at Scale 8

2.1 Sensitivity labels already exist and are published *(hard prerequisite)* . . . 8

2.2 Licensing for automatic labeling (E5 / E5 Compliance) . . . 8

2.3 Information Protection role (least privilege) . . . 9

2.4 Security & Compliance PowerShell connectivity . . . 9

2.5 A classification scheme and the match conditions . . . 9

2.6 In-scope workloads and a pilot location subset . . . 10

2.7 A simulation-first commitment and change control . . . 10

2.8 Behavior toward existing / manually-set labels agreed . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 12

3. Implementation Guide — Automatic Sensitivity Labeling at Scale 13

Step 1 — Assess the auto-labeling posture (Assess) . . . 13

Step 2 — Design the match conditions (Design) . . . 13

Step 3 — Configure client-side auto-labeling (Configure, -WhatIf first) . . . 14
Step 4 — Create service-side auto-labeling **in simulation** — SharePoint/OneDrive (Configure / simulate) . . . 14
Step 5 — Create service-side auto-labeling **in simulation** — Exchange (Configure / simulate) . 14
Step 6 — Let simulation run, then validate precision (Pilot / Validate) . . . 14
Step 7 — Enforce on the pilot location subset (Pilot → Expand) . . . 15
Step 8 — Expand the scope tenant-wide (Expand / enforce at scale) . . . 15
5. Testing & Validation Guide — Automatic Sensitivity Labeling at Scale 16
Test 1 — Target labels exist and are publishable . . . 16
Test 2 — Client-side auto-labeling recommends correctly . . . 16
Test 3 — Service-side policy is created in simulation (not enforcing) . . . 16
Test 4 — Simulation has scanned and produced a match set . . . 17
Test 5 — Simulation precision is acceptable (the enforcement gate) . . . 17
Test 6 — Existing/manual labels are handled as intended . . . 17
Test 7 — Enforcement on the pilot subset matches the accepted simulation . . . 17
Test 8 — Scope and policy health are green . . . 17
Test 9 — Tenant-wide labeling coverage is climbing (at scale) . . . 18
Test 10 — Rollback works . . . 18
6. Troubleshooting Guide — Automatic Sensitivity Labeling at Scale (Microsoft Purview) 18
The target label isn't found by the scripts . . . 18
`Get-AutoSensitivityLabelPolicy` / `New-AutoSensitivityLabelPolicy` is not available . . . 19
A service-side policy is already `Enable` before sign-off . . . 19
Simulation has produced no match set yet . . . 19
Simulation precision is poor — too many false positives (or misses) . . . 19
Client-side auto-labeling isn't recommending in Office . . . 20
Enforced labels don't match what simulation predicted . . . 20
The policy is enforcing on the wrong locations . . . 20
Auto-labeling overwrote (or failed to upgrade) an existing manual label . . . 20
Rollback ran but content is still labeled . . . 20
Coverage is flat or zero after tenant-wide expansion . . . 21
7. Best Practices — Automatic Sensitivity Labeling at Scale (Microsoft Purview) 21
8. Deliverables Summary — Automatic Sensitivity Labeling at Scale (Microsoft Purview) 23
Documentation . . . 23
PowerShell scripts . . . 23
What the buyer receives . . . 24
Outcome . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — Automatic Sensitivity Labeling at Scale (Microsoft Purview)

The problem

Manual sensitivity labeling never scales and never stays consistent. Users forget, guess, or pick the easiest label, and the millions of files already sitting in SharePoint and OneDrive were never labeled at all. Because **DLP, encryption, and retention all key off the label being present and correct**, unlabeled and mislabeled content is a direct compliance gap and data-risk exposure. Yet turning on automatic labeling carelessly is its own hazard: aggressive rules mislabel content and — where labels encrypt — apply **surprise encryption** at scale, generating a ticket flood and eroding trust in days.

The solution

This solution configures Microsoft Purview's two automatic-labeling engines and rolls them out the safe way. **Client-side** auto-labeling recommends the right label inside the Office apps as users create content; **service-side** auto-labeling reaches **existing data at rest** (SharePoint, OneDrive) and **mail in transit** (Exchange). Critically, every service-side policy is **created in simulation** — it scans and reports exactly what *would* be labeled, with **zero** impact — so precision is measured and tuned with the data owner **before** anything is enforced. Enforcement then expands by **location**: a pilot subset first, validated against the accepted simulation, then tenant-wide **at scale**.

The engagement follows the standard, defensible model — **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce at scale)** — with explicit pass/fail gates and a documented rollback at every step.

What's included

- **Full documentation set** — Solution Overview, Requirements Guide, Implementation Guide, Testing Guide (ten pass/fail tests with the precision **enforcement gate**), Troubleshooting, and Best Practices.
- **Five PowerShell tools** — Discovery, the simulation-first configuration/remediation helper, Simulation Report, labeling-coverage Reporting, and a Scope/Health check — all safe-by-default (read-only or `-WhatIf`/simulation unless `-Commit`), using only documented Security & Compliance PowerShell cmdlets, with **no** stored credentials.

Scope boundary and deferrals

This build automates the **application** of sensitivity labels that **already exist and are published** in your tenant; designing and publishing the label taxonomy is a prerequisite. **Which** content is sensitive and **which** label it carries is a classification/records determination owned by your compliance, records, privacy, and data-owner functions — this solution provides the technical method and the evidence, not the determination.

Licensing note

Automatic sensitivity labeling (client- and service-side) is a **premium** capability that typically requires **Microsoft 365 E5** or the **E5 Compliance** add-on, and the Purview portal is evolving. Availability and portal paths are flagged **"verify your licensing/tenant"** rather than assumed.

The bottom line

Data labeled **automatically, consistently, and without the user** — proven in simulation, enforced by stages, and validated — so classification is finally reliable enough for DLP, encryption, and retention to depend on.

1. Solution Overview — Automatic Sensitivity Labeling at Scale (Microsoft Purview)

What this solution is

This solution labels data **automatically, without the user** — it configures Microsoft Purview's two automatic sensitivity-labeling engines, runs them in **simulation first**, tunes them for precision, and then enforces them **at scale** across the tenant. It covers:

- **Client-side auto-labeling** — labels applied (or *recommended*) inside the Office apps

(Word/Excel/PowerPoint/Outlook) as a user creates or edits content, driven by conditions configured on the sensitivity **label** and surfaced through a label policy.

- **Service-side auto-labeling** — labels applied by the service to **data at rest** (SharePoint,

OneDrive) and **data in transit** (Exchange email), configured as an **auto-labeling policy** that is **created in simulation** (it scans and reports what *would* be labeled) and only enforced after its precision is signed off.

The whole engagement is a repeatable, safe rollout: assess what already exists, configure auto-labeling in **simulation**, validate the match precision with the data owner, enforce on a **pilot location subset**, then **expand the location scope to the whole tenant** — using only the Microsoft Purview portal and Security & Compliance PowerShell (no third-party tooling).

> ***Scope boundary.*** *This solution automates the *application* of sensitivity labels that **already exist and are published** in your tenant. Designing the label taxonomy and creating/publishing the labels themselves (and any encryption/marking they carry) is a **prerequisite** (see §2.1) — this build makes those labels apply themselves accurately and at scale.*

Why this matters

- **Manual labeling never scales or stays consistent.** Users forget, guess, or pick the easiest label. Unlabeled and mislabeled content is a compliance gap and a data-risk exposure — DLP, encryption, and retention all key off the label being present and correct.
- **Auto-labeling is the only way to cover data at rest.** Client-side labeling only touches content a user actively edits; the millions of existing files in SharePoint/OneDrive and mail in Exchange are reached only by **service-side** auto-labeling.

• **Aggressive auto-labeling breaks trust fast.** Mislabeling — and, where labels encrypt, *surprise encryption* — generates a ticket flood. **Simulation-first** is what makes labeling at scale safe: you see exactly what *would* be labeled, measure precision, and tune before a single item is touched.

• **Consistency is the point.** A tuned auto-labeling policy applies the same classification the same way across every workload, every day — something a human workforce cannot do.

The two engines (and where each applies)

	Client-side auto-labeling	Service-side auto-labeling
Runs in	Office apps on the client (Word/Excel/PowerPoint/Outlook)	The Microsoft 365 service
Reaches	Content the user is **actively editing**	**Data at rest** (SharePoint, OneDrive) + **in transit** (Exchange)
Behavior	*Recommend* or *automatically apply* a label	Apply a label to matching items in scope
Configured on	The **label** (auto-labeling conditions), surfaced via a **label policy**	An **auto-labeling policy** + rule (`New-AutoSensitivityLabelPolicy` / `New-AutoSensitivityLabelRule`)
Simulation?	No dedicated simulation — piloted via label policy scope	**Yes — created in simulation** (`TestWithoutNotifications`) and validated before enforce
Reaches existing content?	No — new/edited only	**Yes** — sweeps existing content

Together they cover **new content at creation** (client-side) and **existing + in-transit content** (service-side). Both are used in this solution.

Licensing (verify your licensing/tenant)

Automatic sensitivity labeling — **both** client-side automatic labeling in Office apps **and** service-side auto-labeling for SharePoint/OneDrive/Exchange — is a **premium** capability that typically requires **Microsoft 365 E5** or the **E5 Compliance** add-on. Manual labeling is available more broadly, but the automation in this solution is E5-tier. **Verify your licensing/tenant** — Microsoft's licensing tables are the authority; this solution flags the gate rather than assuming it.

Naming convention (used in every doc and script)

Every artifact this solution creates is named `AutoLabel <Type> – <Label> – <Workload/Stage>` and reused verbatim across the docs, scripts, and portal. The **label being applied** is an existing published label — `Confidential \ All Employees` is used as the running example.

Artifact	Name
Service-side auto-label policy — SharePoint/OneDrive (simulation)	`AutoLabel Sim – Confidential – SPO+ODB`
Service-side auto-label rule — SharePoint/OneDrive	`AutoLabel Rule – Confidential – SPO+ODB`

Artifact	Name
Service-side auto-label policy — Exchange (simulation)	`AutoLabel Sim – Confidential – Exchange`
Service-side auto-label rule — Exchange	`AutoLabel Rule – Confidential – Exchange`
Label being auto-applied (existing, published)	`Confidential \ All Employees`
Pilot location subset (SPO)	a named subset of sites; broadened to `All` after sign-off

*Portal artifact names may not accept every punctuation character and the PowerShell examples use an ASCII hyphen (`AutoLabel Sim - Confidential - SPO+ODB`) to avoid encoding issues — the *identity* is what matters; keep it consistent across your tenant.*

What you can control

• **Which existing label** each auto-labeling policy applies (e.g. `Confidential \ All Employees`).

• **The match conditions** — sensitive information types, keywords, or **trainable classifiers** — on each auto-labeling rule, and the **minimum instance / confidence** thresholds that drive precision.

• **Which workloads and locations** are in scope (all vs a pilot subset of sites/mailboxes) and any **exclusions**.

• **Client-side behavior** — whether a label is *recommended* (user confirms) or *auto-applied* silently, configured on the label.

• **The simulation → enforce transition** — nothing is enforced until the simulation match set is accepted.

Limitations (be honest with buyers)

• **Licensing gates the whole capability.** Automatic labeling (client- and service-side) requires **E5 / E5 Compliance** — **verify your licensing/tenant**; without it this solution cannot run.

• **Labels must already exist.** This build applies published labels automatically; it does not design or create the taxonomy (see §2.1).

• **Service-side scanning has latency and quotas.** Simulation and enforcement scan large volumes over time; results are **not** instantaneous and item/throughput limits apply — **verify current limits for your tenant.**

• **Auto-labeling only *raises* or sets a label under defined rules.** By default the service won't overwrite a higher-priority label a user set manually — behavior around existing labels is configurable and **tenant/version-sensitive; verify in your tenant.**

• **Where the applied label encrypts, auto-labeling encrypts.** Applying an encrypting label at scale is high-impact — this is precisely why simulation and a pilot location subset come first.

• **The portal is evolving.** Portal paths and some cmdlet parameters change; version-sensitive paths are flagged "verify in your tenant."

The standard deployment model

*Assess the current auto-labeling posture and confirm the target labels exist → design the match conditions → configure client-side and service-side auto-labeling in **simulation** → validate match precision with the data owner → enforce on a **pilot location subset** → validate → **expand the scope tenant-wide (enforce at scale).** Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce).***

2. Requirements Guide — Automatic Sensitivity Labeling at Scale

Verify **every** requirement below before starting the Implementation Guide. These are the **solution-specific** prerequisites for standing up automatic sensitivity labeling; they build on the shared **Global Requirements — Microsoft Purview Solutions** (compliance licensing, least-privilege roles, Security & Compliance PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, and a pilot scope with change control). Read the Global Requirements first; this guide does not repeat them.

***Deferral note.** *Which* content is sensitive, and *which* label it should carry, is a **classification/records determination** owned by your compliance, records, privacy, and data owners — not a technical default. This solution provides the **method to apply labels accurately and at scale, and the evidence** that it does; it does not decide the classification scheme.*

2.1 Sensitivity labels already exist and are published *(hard prerequisite)*

• **Description:** The labels you intend to auto-apply (e.g. `Confidential \ All Employees`) must already be **created and published** to users via a sensitivity label policy, with any encryption/content-marking already configured and validated. This solution automates the *application* of existing labels — it does not create the taxonomy.

• **Why required:** An auto-labeling policy can only apply a label that exists; if the label encrypts, that protection must already be proven to enforce before you apply it at scale.

• **How to verify:** `Get-Label` lists the target label; `Get-LabelPolicy` shows it is published; a user can see and manually apply it in Office.

• **How to implement if missing:** Design, build, and publish the label taxonomy first (a separate labels-implementation effort). Do not proceed until the target labels exist and — if they encrypt — encryption is proven to enforce.

2.2 Licensing for automatic labeling (E5 / E5 Compliance)

• **Description: Automatic** sensitivity labeling — **client-side** automatic labeling in Office apps **and service-side** auto-labeling for SharePoint/OneDrive/Exchange — is a **premium** capability that typically requires **Microsoft 365 E5** or the **E5 Compliance** add-on. Manual labeling is available more broadly; the automation in this solution is E5-tier.

• **Why required:** The auto-labeling policies and client-side auto-labeling conditions in this solution are not available without the right plan.

• **How to verify:** M365 admin center → subscriptions/add-ons; confirm **Information Protection → Auto-labeling** is visible in the Purview portal and that `New-AutoSensitivityLabelPolicy` is available in Security & Compliance PowerShell. **Verify your licensing/tenant** — Microsoft's licensing tables are the authority.

• **How to implement if missing:** Assign/purchase E5 or E5 Compliance for at least the pilot population before configuring auto-labeling.

2.3 Information Protection role (least privilege)

• **Description:** The Purview role group this solution needs is **Information Protection Admin** (create/manage sensitivity labels, label policies, and **auto-labeling** policies/rules). Read-only assessment needs **Information Protection Reader** or **Information Protection Analyst**. Global Administrator is **not** required for day-to-day auto-labeling work.

• **Why required:** The auto-labeling cmdlets (`New-AutoSensitivityLabelPolicy`, `New-AutoSensitivityLabelRule`, `Set-AutoSensitivityLabelPolicy`) and `Set-Label` are gated by this role group.

• **How to verify:** Purview portal → **Roles & scopes** → **Permissions**; confirm the admin account is in *Information Protection* (or *Information Protection Admin*).

• **How to implement if missing:** Add the admin to the least-privilege role group under change control; wait for role propagation before running cmdlets.

2.4 Security & Compliance PowerShell connectivity

• **Description:** A workstation with the **ExchangeOnlineManagement** module (which provides `Connect-IPPSSession` for Security & Compliance PowerShell) and, where Graph is used for evidence, **Microsoft Graph PowerShell** — with a modern-auth / MFA-capable admin account.

• **Why required:** All five provided scripts and every configuration cmdlet run through `Connect-IPPSSession`.

• **How to verify:** `Connect-IPPSSession` succeeds and `Get-AutoSensitivityLabelPolicy` returns without error.

• **How to implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm modern-auth sign-in. See the Global Requirements for the full toolchain.

2.5 A classification scheme and the match conditions

• **Description:** An agreed **classification scheme** that maps *what makes content sensitive* to a label — the **sensitive information types (SITs)**, **keywords**, or **trainable classifiers** that will drive each auto-labeling rule, and the **minimum instance count / confidence** thresholds. For example: "apply `Confidential \ All Employees` where content contains ≥ N instances of a financial-account SIT."

• **Why required:** Auto-labeling precision is entirely a function of the rule conditions; without an agreed scheme the policy is arbitrary and will misfire.

• **How to verify:** Confirm the priority SITs/classifiers exist in Purview → **Data classification → Sensitive info types / Trainable classifiers**; agree the thresholds with the data owner.

• **How to implement if missing:** Define the conditions and thresholds (and create any custom SITs/classifiers) before authoring rules. Prefer a small number of high-precision conditions to start.

2.6 In-scope workloads and a pilot location subset

• **Description:** The **workloads in scope** (SharePoint, OneDrive, Exchange for service-side; the Office apps for client-side) and a **pilot location subset** — a named set of sites/mailboxes to enforce on first before expanding to `All`. Service-side auto-labeling location scoping can also use **adaptive scopes** where available (**verify your licensing/tenant**).

• **Why required:** Enforcing across the whole tenant at once removes the ability to catch problems cheaply; the house model enforces on a subset first, then expands *at scale*.

• **How to verify:** Confirm the in-scope workloads and the pilot site/mailbox subset with stakeholders.

• **How to implement if missing:** Identify the pilot locations (and the eventual full scope) before creating policies.

2.7 A simulation-first commitment and change control

• **Description:** An explicit commitment that **every service-side auto-labeling policy is created in simulation** (`TestWithoutNotifications`), reviewed for precision with the data owner, and only enforced after sign-off — and that **enforcement expands by location** (pilot subset → tenant) under a change window.

• **Why required:** Auto-labeling at scale is high-impact and (where the label encrypts) hard to reverse cleanly; simulation and staged expansion are the safety controls.

• **How to verify:** Confirm the data owner who will review the simulation match set, the false-positive threshold, a change window, and an approver.

• **How to implement if missing:** Agree the review owner, threshold, and change-control path before creating any policy.

2.8 Behavior toward existing / manually-set labels agreed

• **Description:** A decision on how auto-labeling should behave toward content that **already carries a label** — by default the service does not lower or override a higher-priority label a user set manually, but the treatment of existing labels is configurable and **tenant/version-sensitive**.

• **Why required:** The wrong setting can silently overwrite deliberate manual classification (or fail to upgrade obviously-sensitive content) — both erode trust and must be a conscious choice, confirmed in simulation.

• **How to verify:** Confirm the intended behavior with the data owner; **verify the current setting/behavior in your tenant** and observe it in the simulation results before enforcing.

• **How to implement if missing:** Decide and document the intended behavior; validate it against the simulation match set (Testing Guide) before enforcement.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Automatic Sensitivity Labeling at Scale

Follow these steps **in order**. Configuration is done in the **Microsoft Purview portal** (`https://purview.microsoft.com`) and **Security & Compliance PowerShell** (`Connect-IPPSSession`). Every write is run `-WhatIf` first, every service-side policy is **created in simulation** (`TestWithoutNotifications`) and validated for precision **before** it is enforced, and enforcement is **expanded by location** (pilot subset → tenant). Artifact names follow the `AutoLabel <Type> – <Label> – <Workload/Stage>` convention from the Overview and are reused verbatim. Portal paths are current at authoring — **verify in your tenant**, as the portal is evolving.

> ***Model:** Assess the posture and confirm the labels exist, design the match conditions, configure client- and service-side auto-labeling in **simulation**, validate match precision, enforce on a **pilot location subset**, validate, then **expand tenant-wide (enforce at scale).** Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce). Prerequisite reminder (§2.1):** the labels you auto-apply must already exist and be published. This guide applies them automatically; it does not create them.*

Step 1 — Assess the auto-labeling posture (Assess)

1. Run `IPAUTO-Discovery.ps1` (read-only). It inventories the **existing sensitivity labels** (`Get-Label`) and which ones already carry **client-side auto-labeling** conditions, the **sensitivity label policies** (`Get-LabelPolicy`), and every **service-side auto-labeling policy with its mode** (`Get-AutoSensitivityLabelPolicy`) and its **rules** (`Get-AutoSensitivityLabelRule`).

2. Review the output: confirm the **target label exists and is published** (§2.1), and check whether any auto-labeling is already enforced so you don't collide or double-label.

3. **Gate:** you have a written baseline of existing labels + auto-labeling policies, and confirmed the target label (`Confidential \ All Employees`) exists, is published, and — if it encrypts — its protection already enforces.

Step 2 — Design the match conditions (Design)

1. With the data owner, finalize per target label: the **sensitive info types / keywords / trainable classifiers**, the **minimum instance count and confidence**, and the **in-scope workloads** (SharePoint, OneDrive, Exchange) and any **exclusions** (§2.5, §2.6).

2. Decide the **pilot location subset** (the sites/mailboxes to enforce on first) and the eventual full scope, and the intended **behavior toward existing labels** (§2.8).

3. **Gate:** a signed-off condition sheet exists — label, conditions, thresholds, workloads, pilot subset, existing-label behavior — before any policy is created.

Step 3 — Configure client-side auto-labeling (Configure, -WhatIf first)

1. Client-side auto-labeling is configured **on the label** itself (its "auto-labeling for files and emails" conditions) and surfaces to users through the label policy. Run `IPAUTO-AutoLabelConfig.ps1 -ClientSide -Label "Confidential \ All Employees" -Sit "<SIT name>" -MinCount <N> -Recommend -WhatIf`. It prints the `Set-Label` change it would make — setting the label's client-side auto-labeling conditions to *recommend* (user confirms) rather than silently apply, to start.

2. Review the change, then re-run with `-Commit`. Because it recommends, users are prompted (not surprised); you can switch to auto-apply later once behavior is trusted.

3. In the portal (**Information Protection** → **Labels** → **the label** → **Auto-labeling for files and emails**) confirm the condition is set. **Verify the exact path in your tenant.**

4. **Gate:** the label shows the client-side auto-labeling condition; a pilot user editing matching content sees the label **recommended** (validated in Testing §5).

Step 4 — Create service-side auto-labeling in simulation — SharePoint/OneDrive (Configure / simulate)

1. Run `IPAUTO-AutoLabelConfig.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -RuleName "AutoLabel Rule - Confidential - SPO+ODB" -Label "Confidential \ All Employees" -Workload SharePoint,OneDrive -Sit "<SIT name>" -MinCount <N> -WhatIf`. It shows the `New-AutoSensitivityLabelPolicy` **in simulation mode** (`-Mode TestWithoutNotifications`) plus the `New-AutoSensitivityLabelRule` (the sensitive-info-type conditions) it would create. **No content is labeled in simulation.**

2. Review the `-WhatIf` output — confirm the mode is a `TestWithout...` (simulation) value, the label, the conditions/threshold, and the locations. Re-run with `-Commit` to create the simulation policy + rule.

3. **Gate:** `Get-AutoSensitivityLabelPolicy` shows the policy in a **simulation** mode; the rule exists with the intended conditions; nothing is enforced.

Step 5 — Create service-side auto-labeling in simulation — Exchange (Configure / simulate)

1. If email is in scope, repeat Step 4 for Exchange: `IPAUTO-AutoLabelConfig.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - Exchange" -RuleName "AutoLabel Rule - Confidential - Exchange" -Label "Confidential \ All Employees" -Workload Exchange -Sit "<SIT name>" -MinCount <N> -WhatIf`, review, then `-Commit`.

2. **Gate:** the Exchange simulation policy + rule exist in simulation mode; nothing is enforced.

Step 6 — Let simulation run, then validate precision (Pilot / Validate)

1. Let the simulation scan the in-scope locations — it processes large volumes over time (**verify current throughput/limits for your tenant**).

2. Run `IPAUTO-SimulationReport.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB"`. It confirms the policy is still in **simulation**, reports scan status, and points to the **items matched** — what *would* be labeled — for precision review.

3. Review the matches with the data owner for **false positives** (content that should **not** get the label) and **false negatives** (obvious content missed). If precision is poor, tune the **rule** conditions (tighter/different SITs, higher `-MinCount`, add keywords, or a trainable classifier) via `IPAUTO-AutoLabelConfig.ps1 -ServiceSide ... -UpdateRule -Commit`, and let simulation re-run. Iterate.

4. **Gate:** the simulation match set is **reviewed and accepted by the data owner**; false-positive rate is within the agreed threshold; behavior toward existing labels (§2.8) is confirmed. **Do not enforce until this gate passes.**

Step 7 — Enforce on the pilot location subset (Pilot → Expand)

1. Once simulation is accepted, enforce **on the pilot subset only**: `IPAUTO-AutoLabelConfig.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -Enforce -Locations "<pilot sites>" -WhatIf`, review the `Set-AutoSensitivityLabelPolicy -Mode Enable` (scoped to the pilot locations) it would run, then `-Commit`.

2. Monitor the first enforcement window: run `IPAUTO-SimulationReport.ps1` and `IPAUTO-Reporting.ps1` and confirm labels are being applied on the pilot locations as simulation predicted — no unexpected encryption/marking.

3. **Gate:** enforced labels on the pilot subset match the accepted simulation; no surprises. Rollback: `IPAUTO-AutoLabelConfig.ps1 -ServiceSide -PolicyName "<name>" -BackToSimulation -Commit` returns the policy to simulation (stops labeling).

Step 8 — Expand the scope tenant-wide (Expand / enforce at scale)

1. With the pilot enforcement validated, expand the location scope: `IPAUTO-AutoLabelConfig.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -Enforce -Locations All -WhatIf`, review, then `-Commit`. Repeat for the Exchange policy.

2. Optionally switch **client-side** auto-labeling from *recommend* to *auto-apply* (`IPAUTO-AutoLabelConfig.ps1 -ClientSide -Label "..." -AutoApply -Commit`) once the recommended behavior is trusted.

3. Run `IPAUTO-Reporting.ps1` on a cadence to confirm labeling coverage is climbing as expected, and `IPAUTO-ScopeHealth.ps1` to confirm policy distribution/scope health across the tenant.

4. **Gate:** all §5 tests pass; tenant-wide enforcement confirmed; labeling coverage reported; a rollback path is documented per policy.

Order of operations

`Assess posture + confirm labels exist → Design match conditions → Configure client-side (recommend, -WhatIf → commit) → Create service-side SIMULATION`

`policies (SPO/ODB, then Exchange) → Let simulation run → Validate precision → Enforce on PILOT location subset → Validate → Expand scope tenant-wide (enforce at scale) → Report + health-check`

5. Testing & Validation Guide — Automatic Sensitivity Labeling at Scale

Run these tests after the Implementation Guide, **in order**. Each has an explicit **pass/fail** signal and a rollback. Do not enforce a service-side policy, and do not expand from the pilot location subset, until the relevant tests pass. All checks use the Purview portal and Security & Compliance PowerShell — the provided scripts automate the read-only ones.

> ***Reminder:*** *you are validating that auto-labeling *behaves as specified and applies the right label accurately* — not deciding which content is sensitive (that classification is the data owner's determination, §2 deferral).*

Test 1 — Target labels exist and are publishable

- **How:** `IPAUTO-Discovery.ps1`, or `Get-Label | Format-Table DisplayName,Name` and `Get-LabelPolicy`.
- **Pass:** the label to be auto-applied (`Confidential \ All Employees`) exists, is published, and — if it encrypts — its protection already enforces (§2.1).
- **Fail:** label missing or unpublished → stop; complete the labels prerequisite before auto-labeling.

Test 2 — Client-side auto-labeling recommends correctly

- **How:** as a **pilot** user, create a document/email containing content that matches the rule (e.g. test data with the configured SIT). Observe whether the label is **recommended**.
- **Pass:** the intended label is recommended for matching content and **not** for clearly non-matching content; the user is prompted (not silently overridden).
- **Fail:** over- or under-recommending → tune the label's client-side auto-labeling conditions (`Set-Label` via `IPAUTO-AutoLabelConfig.ps1 -ClientSide ... -Commit`) and re-test.

Test 3 — Service-side policy is created in simulation (not enforcing)

- **How:** `IPAUTO-SimulationReport.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB"` (and the Exchange policy); confirm **Mode** is a `TestWithout/WithNotifications` (simulation) value, not `Enable`.
- **Pass:** each policy is in **simulation**, has begun scanning its in-scope locations, and has **labeled nothing**.
- **Fail:** a policy is already `Enable` before sign-off → return it to simulation (`IPAUTO-AutoLabelConfig.ps1 -ServiceSide ... -BackToSimulation -Commit`) and investigate why.

Test 4 — Simulation has scanned and produced a match set

• **How:** `IPAUTO-SimulationReport.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB"`; check scan/deployment status and the matched-items pointer.

• **Pass:** the simulation has progressed and there is a reviewable match set (what *would* be labeled).

• **Fail:** no results yet → allow scan time (service-side scanning is not instant and is subject to limits — **verify limits for your tenant**); confirm the in-scope locations actually contain matching content.

Test 5 — Simulation precision is acceptable (the enforcement gate)

• **How:** review the simulation match set with the **data owner**; sample matched items for false positives and spot-check obvious sensitive content for false negatives.

• **Pass:** the false-positive rate is within the agreed threshold **and** the data owner accepts the match set; behavior toward already-labeled content (§2.8) is as intended.

• **Fail:** too many false positives / misses → tune the rule conditions (`-UpdateRule -Commit`), re-run simulation, re-review. **This is the gate before any enforcement.**

Test 6 — Existing/manual labels are handled as intended

• **How:** in the simulation match set, inspect items that **already carry a label**. Confirm the policy's behavior toward them matches the §2.8 decision (e.g. it does not lower a higher-priority manual label).

• **Pass:** already-labeled content is treated exactly as decided; no deliberate manual classification is silently overwritten.

• **Fail:** unexpected override/no-op → adjust the behavior setting (**verify the setting in your tenant**), re-simulate, re-review.

Test 7 — Enforcement on the pilot subset matches the accepted simulation

• **How:** after enforcing on the pilot locations (Step 7), run `IPAUTO-SimulationReport.ps1` and `IPAUTO-Reporting.ps1` and spot-check newly labeled items on those locations.

• **Pass:** enforced labels on the pilot subset match what simulation predicted; no unexpected encryption or content marking on user content.

• **Fail:** divergence or surprise protection → roll the policy back to simulation (`-BackToSimulation -Commit`) and investigate before expanding.

Test 8 — Scope and policy health are green

• **How:** `IPAUTO-ScopeHealth.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB"`.

• **Pass:** the policy's locations resolve as intended (pilot subset, or `All` after expansion), no unintended inclusions/exclusions, and distribution/deployment status is healthy.

• **Fail:** wrong locations or a distribution error → correct the scope (`-Enforce -Locations ...`) or investigate the distribution status; re-check.

Test 9 — Tenant-wide labeling coverage is climbing (at scale)

- **How:** after expanding to `All` (Step 8), run `IPAUTO-Reporting.ps1` on a cadence to report labeled-item activity/coverage.
- **Pass:** the volume of auto-applied labels grows and stabilizes across workloads, consistent with the accepted simulation precision — content is being labeled **without the user**.
- **Fail:** flat/zero coverage → confirm enforcement mode, locations, and that matching content exists; re-check scan limits for your tenant.

Test 10 — Rollback works

- **How:** on a test policy, run `IPAUTO-AutoLabelConfig.ps1 -ServiceSide -PolicyName "<name>" -BackToSimulation -Commit`.
- **Pass:** the policy returns to simulation and stops labeling; existing labels already applied remain (auto-labeling does not un-label on rollback) — confirm this is understood.
- **Fail:** policy stays enforcing → verify the `Set-AutoSensitivityLabelPolicy -Mode` change and role/permissions.

Exit criteria (all must be true before tenant-wide enforcement)

`Target labels exist + publishable · client-side recommends accurately · service-side created in SIMULATION · simulation scanned + match set produced · precision accepted by data owner · existing-label behavior correct · pilot enforcement == accepted simulation · scope/health green · coverage climbing at scale · rollback proven`

6. Troubleshooting Guide — Automatic Sensitivity Labeling at Scale (Microsoft Purview)

Field issues and how to resolve them, in roughly the order you are likely to hit them. Every check here is safe: read-only (`IPAUTO-Discovery.ps1`, `IPAUTO-Validate.ps1`, `IPAUTO-Report.ps1`, `IPAUTO-ScopeHealth.ps1`), a `-WhatIf`/preview (`IPAUTO-Configure.ps1` without `-Commit`), or a controlled change on the pilot scope. Nothing here labels content by surprise. When a portal path, mode value, or cmdlet parameter differs from this guide, **verify it in your tenant** — automatic labeling is E5/E5 Compliance and the portal is evolving.

The target label isn't found by the scripts

Symptom: `IPAUTO-Discovery.ps1` or `IPAUTO-Configure.ps1` reports the target label (`Confidential \ All Employees`) as NOT found, and refuses to build a policy. **Cause:** Auto-labeling can only apply an **existing, published** label (§2.1). The label may not exist, may be unpublished, or you may be passing the display name where the internal `Name`/GUID is expected. **Fix:** Run `IPAUTO-Discovery.ps1 -TargetLabel "Confidential \ All Employees"` and read the labels list; confirm with `Get-Label` and `Get-LabelPolicy` that the label exists and is published. If it is missing,

complete the labels prerequisite first (a separate labels-implementation effort) — do not proceed. Pass the exact name the inventory shows.

`Get-AutoSensitivityLabelPolicy` / `New-AutoSensitivityLabelPolicy` is not available

Symptom: A script prints "Get-AutoSensitivityLabelPolicy not available — check licensing/role", or the cmdlet errors. **Cause:** Either you are not connected to Security & Compliance PowerShell, or the account/tenant lacks the plan/role for **automatic** labeling (E5 / E5 Compliance; **Information Protection Admin** role — §2.2/§2.3). **Fix:** `Connect-IPPSSession` with a modern-auth admin; confirm the account is in the Information Protection role group and the tenant is licensed for auto-labeling. Then re-run `IPAUTO-ScopeHealth.ps1 -PolicyName "<name>"` — its readiness flags confirm the cmdlets resolve. Microsoft's licensing tables are the authority — **verify your licensing/tenant.**

A service-side policy is already `Enable` before sign-off

Symptom: `IPAUTO-Validate.ps1 -PolicyName "<name>" -ExpectMode Simulation` returns **FAIL** on the "Mode is SIMULATION" check — the policy is enforcing when it should still be simulating. **Cause:** The policy was enforced (or created) outside the simulation-first flow, so it is labeling content before the match set was accepted (§2.7). **Fix:** Return it to simulation immediately: `IPAUTO-Configure.ps1 -ServiceSide -PolicyName "<name>" -BackToSimulation -Commit` (runs `Set-AutoSensitivityLabelPolicy -Mode TestWithoutNotifications`). Note this **stops new labeling but does not un-label** already-labeled content. Then investigate who enforced it and re-run `IPAUTO-Validate.ps1`.

Simulation has produced no match set yet

Symptom: `IPAUTO-Validate.ps1` flags "Simulation scan/match signal present" as not yet satisfied, or `IPAUTO-Report.ps1` shows a blank/zero match count. **Cause:** Service-side scanning is **not instant** and is subject to item/throughput limits; the scan may still be running, or the in-scope locations may contain no matching content. **Fix:** Allow scan time and re-run `IPAUTO-Report.ps1 -PolicyName "<name>"`. Confirm the in-scope locations actually hold content that matches the rule (SIT + `-MinCount`). A zero count often means "still scanning", not "nothing matches" — **verify current scan limits/latency for your tenant** before concluding the policy is broken.

Simulation precision is poor — too many false positives (or misses)

Symptom: In the `IPAUTO-Report.ps1` match set, the data owner sees content that should **not** get the label (false positives) or obvious sensitive content missed (false negatives). **Cause:** The rule conditions are too broad, too narrow, or built on a thin/noisy sensitive-information type — precision is entirely a function of the conditions (§2.5). **Fix:** Tune the rule and re-simulate: `IPAUTO-Configure.ps1 -ServiceSide -PolicyName "<name>" -RuleName "<rule>" -UpdateRule -Sit "<better SIT>" -MinCount <higher N> -Commit` (raise `-MinCount`, use a higher-confidence or more specific SIT, add keywords, or a trainable classifier). Let simulation re-run, then re-review with `IPAUTO-Report.ps1` and `IPAUTO-Validate.ps1`. **This is the gate before any enforcement (Testing Test 5).** Which content is truly sensitive remains the data owner's determination.

Client-side auto-labeling isn't recommending in Office

Symptom: A pilot user editing matching content is not prompted with the label; `IPAUTO-Configure.ps1 -ClientSide` appeared to run. **Cause:** The label's client-side auto-labeling condition wasn't set, the user's label policy doesn't publish that label, Office needs to refresh its policy, or the Office build is too old for the feature. **Fix:** Confirm the condition in the portal (**Information Protection** → **Labels** → **the label** → **Auto-labeling for files and emails** — **verify this path in your tenant**). Confirm the label is published to that user (`Get-LabelPolicy`). Have the user restart Office to refresh policy, and confirm a supported Office build. Note that `IPAUTO-Configure.ps1 -ClientSide` deliberately previews/`-WhatIf`s the `Set-Label` change because the **exact auto-labeling parameter surface is version-sensitive** — supply your tenant's confirmed parameters to actually apply it, then re-test.

Enforced labels don't match what simulation predicted

Symptom: After enforcing on the pilot subset, spot-checks show labels applied to items simulation did not flag (or vice-versa). **Cause:** The rule or scope changed between the accepted simulation and enforcement, or scan drift/latency means the two snapshots differ. **Fix:** Roll back first — `IPAUTO-Configure.ps1 -ServiceSide -PolicyName "<name>" -BackToSimulation -Commit` — then compare the current rule against the accepted design, re-simulate, and re-validate with `IPAUTO-Report.ps1` before re-enforcing. Do not expand scope until pilot enforcement equals the accepted simulation (Testing Test 7).

The policy is enforcing on the wrong locations

Symptom: `IPAUTO-ScopeHealth.ps1 -PolicyName "<name>"` shows `ALL — tenant-wide` when you intended a pilot subset, or the wrong sites/mailboxes are targeted. **Cause:** The location/scope parameters were set to `All` (or the wrong subset), or an adaptive scope resolves more broadly than expected. Location parameter names are **tenant/version-sensitive**. **Fix:** Re-scope with `IPAUTO-Configure.ps1 -ServiceSide -PolicyName "<name>" -Enforce -Locations "<pilot site urls>" -WhatIf`, review the previewed `Set-AutoSensitivityLabelPolicy` change, then `-Commit`. Re-run `IPAUTO-ScopeHealth.ps1` and confirm the locations resolve to the intended subset. **Verify the current location/adaptive-scope parameter names in your tenant** — `IPAUTO-ScopeHealth.ps1` flags any scope property it cannot resolve rather than asserting one exists.

Auto-labeling overwrote (or failed to upgrade) an existing manual label

Symptom: Content a user had manually labeled shows a different label after enforcement, or obviously sensitive already-labeled content wasn't upgraded. **Cause:** The policy's behavior toward existing/manually-set labels (§2.8) is not set as intended — this behavior is configurable and **tenant/version-sensitive**. **Fix:** Confirm the intended behavior with the data owner, set it on the policy (**verify the current setting/parameter in your tenant**), and re-validate it against the **simulation** match set before re-enforcing — inspect items that already carry a label in `IPAUTO-Report.ps1` output (Testing Test 6). Whether a deliberate manual classification may be changed is a compliance/data-owner decision, not a technical default.

Rollback ran but content is still labeled

Symptom: You ran `-BackToSimulation -Commit` but previously labeled items still carry the label. **Cause:** This is expected — returning a policy to simulation **stops new labeling; it does not un-label** content already labeled while enforced. **Fix:** Confirm the policy is back in simulation with `IPAUTO-Validate.ps1 -PolicyName "<name>" -ExpectMode Simulation`. If specific items must be re-labeled or cleared, that is a separate, deliberate action on those items (and, where the label encrypts, an access decision) — treat it as change-controlled, not a side effect of rollback.

Coverage is flat or zero after tenant-wide expansion

Symptom: After expanding to `All`, `IPAUTO-Report.ps1` shows label counts not climbing. **Cause:** The policy may not actually be in `Enable`, the locations may not have expanded, matching content may be sparse, or the scan is still catching up. **Fix:** Run `IPAUTO-Validate.ps1 -PolicyName "<name>" -ExpectMode Enforce` (confirms `Enable`) and `IPAUTO-ScopeHealth.ps1` (confirms locations resolve to `All`). Confirm matching content exists and allow scan time; **re-check scan limits for your tenant.** Report on a cadence rather than judging from a single snapshot.

When to stop and escalate

- **Auto-labeling cmdlets/paths aren't available** (`New-AutoSensitivityLabelPolicy` missing, portal

auto-labeling node absent) → this is a **licensing/tenant** action (E5 / E5 Compliance, Information Protection role). Record it and do not force a path that isn't there — **verify your licensing/tenant.**

- A question of **which content is sensitive, which label it should carry, or whether a manual label may

be changed **is being asked of you** → **escalate to the** data owner / compliance / records / privacy** function. This solution provides the technical method to apply labels accurately and the evidence that it does — never the classification determination (§2 deferral).

- **A mode value, location parameter, or Set-Label auto-labeling parameter in this guide doesn't match

your environment** → the Purview auto-labeling surface is evolving; confirm the current cmdlet/parameter against Microsoft's documentation and your tenant rather than assuming the step is wrong. The scripts never invent parameters — they preview the change and apply only what you confirm.

7. Best Practices — Automatic Sensitivity Labeling at Scale (Microsoft Purview)

- **Simulate before you label — always.** Every service-side auto-labeling policy is created in

simulation (`TestWithoutNotifications`) and reviewed for precision **before** it ever touches content. Simulation is the single control that makes labeling at scale safe: you see exactly what *would* be labeled, measure the false-positive rate, and tune — with **zero** impact on users. The configuration helper defaults to `-WhatIf` for the same reason.

- **Start with a few high-precision conditions.** A rule with one strong sensitive-information type at a

sensible `-MinCount` and confidence beats a broad rule that fires everywhere. Precision first, coverage second — you can always widen conditions once the match set is trusted.

- **Recommend before you auto-apply (client-side).** Configure client-side auto-labeling to

recommend first so users are prompted, not surprised. Switch to silent **auto-apply** only after the recommended behavior is trusted and the false-positive rate is low.

- **Enforce by location, not all at once.** After simulation is accepted, enforce on a **pilot location

subset** first, validate that enforced labels match the accepted simulation, and only then expand to `All`. Staged expansion keeps the blast radius small and the rollback cheap.

- **Respect encryption as high-impact.** Where the applied label **encrypts**, applying it at scale

changes access to content and is hard to reverse cleanly. Confirm in simulation exactly which items would be encrypted, and treat the first enforcement window with extra care.

- **Decide existing-label behavior deliberately (§2.8).** By default the service will not lower a

higher-priority label a user set manually — but the behavior is configurable and **tenant/version-sensitive**. Make it a conscious decision with the data owner and **verify it in the simulation match set** before enforcing, so deliberate manual classification is never silently overwritten.

- **The classification determination is the data owner's, not yours.** *Which* content is sensitive and

which label it carries is a compliance/records/data-owner call. You provide the technical method to apply the label accurately and the evidence that it does — you do not decide the scheme.

- **Keep the levers healthy.** Auto-labeling precision is entirely a function of the sensitive

information types, keywords, and trainable classifiers behind each rule. A thin or noisy taxonomy caps how precisely you can label; invest in the SITs/classifiers before blaming the policy.

- **Allow for scan latency and limits.** Service-side simulation and enforcement scan large volumes over

time — results are **not** instantaneous and item/throughput limits apply. **Verify current limits for your tenant** and give the scan time before concluding a policy is broken.

- **Keep the rollback one command away.** Every enforced policy should be one change

(`-BackToSimulation -Commit`) from returning to simulation. Note that rolling back **stops new labeling but does not un-label** already-labeled content — make sure that is understood before you enforce.

- **Report on a cadence.** Run the read-only `Discovery` → `SimulationReport` → `Reporting` → `ScopeHealth`

chain regularly so labeling coverage and policy health stay visible as data and adoption drift, not measured once at go-live.

- **Verify availability, never assume it.** Automatic labeling (client- and service-side), adaptive

scopes for location targeting, and some cmdlet parameters are licensing-dependent and evolving. When a path, mode value, or parameter differs from this guide, confirm it in your tenant and use the current equivalent — do not force a path that is not there.

8. Deliverables Summary — Automatic Sensitivity Labeling at Scale (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case, scope boundary, and what's in the box
1. Solution Overview	What this delivers (client-side + service-side automatic labeling), the two engines and where each applies, the simulation-first model, the licensing caveat, the scope boundary (applies existing published labels), and the naming convention
2. Requirements Guide	Every solution-specific prerequisite with Verify / Implement-if-missing (labels already published, E5/E5 Compliance licensing, Information Protection role, S&C PowerShell, match conditions, in-scope workloads + pilot subset, simulation-first commitment, existing-label behavior)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + PowerShell steps: assess posture, design conditions, configure client-side (recommend), create service-side SIMULATION policies (SPO/ODB then Exchange), let simulation run, validate precision, enforce on pilot subset, expand tenant-wide, report + health-check — each step with pass/fail gates and rollback
5. Testing Guide	Ten validation procedures with explicit pass/fail, the precision **enforcement gate**, existing-label behavior, and proven rollback
6. Troubleshooting Guide	Field issues (labels missing, condition not visible, simulation empty, surprise encryption, coverage flat) and remediation
7. Best Practices	Field-proven recommendations for labeling accurately and safely at scale

PowerShell scripts

All are safe-by-default (read-only, or `-WhatIf` / service-side **simulation** mode unless `-Commit`), use `Connect-IPPSSession` (Security & Compliance PowerShell) and/or `Connect-MgGraph`, use only documented cmdlets, and store **no** credentials.

Script	What it does	
`IPAUTO-Discovery.ps1`	Read-only inventory of existing sensitivity **labels** (and which carry client-side auto-labeling), **label policies**, and every **service-side auto-labeling policy + rule with its mode**. Confirms the target label exists/is published. Writes `AutoLabel Posture – Estate – Discovery`	
`IPAUTO-AutoLabelConfig.ps1`	Configuration/remediation helper. Defaults to `-WhatIf`. `-ClientSide` sets a label's client-side auto-labeling conditions (recommend/auto-apply via `Set-Label`); `-ServiceSide` creates a service-side policy **in simulation** (`New-AutoSensitivityLabelPolicy -Mode TestWithoutNotifications`) + rule; `-Enforce -Locations <pilot	All> flips a validated policy to Enable on a location scope; -BackToSimulation rolls back; -UpdateRule tunes conditions. Every write is gated behind -Commit`	
`IPAUTO-SimulationReport.ps1`	Read-only. Confirms a service-side policy is still in **simulation**, reports scan/deployment status, and points to the **matched items** (what *would* be labeled) for precision review. Writes `AutoLabel Simulation – <policy> – Review`	
`IPAUTO-Reporting.ps1`	Read-only labeling-coverage rollup from Purview auditing (label-applied activity by workload) to confirm coverage is climbing after enforcement. Writes `AutoLabel Coverage – Estate – Report`	
`IPAUTO-ScopeHealth.ps1`	Read-only scope/health helper: resolves a policy's **locations** (pilot subset vs `All`), reports distribution/deployment status, and confirms sign-in + licensing surface. Writes `AutoLabel Scope Health – <policy> – Check`	

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce at scale).**

Outcome

Sensitive content labeled **automatically and consistently, without the user** — client-side labeling recommending the right label as content is created, and tuned service-side auto-labeling sweeping existing data at rest and mail in transit — every service-side policy proven in **simulation** and enforced by **location subset first, then tenant-wide**. Classification, records, and legal determinations remain with the organization's governance functions; this solution delivers the technical method and the evidence that labels apply accurately and at scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPAUTO-Configure.ps1

Purpose: Configures Microsoft Purview automatic sensitivity labeling: the CLIENT-SIDE auto-labeling conditions on a sensitivity label (recommend or auto-apply), and SERVICE-SIDE auto-labeling policies + rules for SharePoint/OneDrive/Exchange. Every service-side policy is created in SIMULATION (-Mode TestWithoutNotifications) so nothing is labeled until you review and enforce. SAFE BY DEFAULT: with no -Commit it only PREVIEWS the exact cmdlet it would run (-WhatIf); it changes nothing. Writes (New-/Set-) run only when -Commit is supplied. HOW TO RUN PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. Preview a service-side SIMULATION policy (no change): .\IPAUTO-Configure.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" `-RuleName "AutoLabel Rule - Confidential - SPO+ODB" -Label "Confidential \ All Employees"` -Workload SharePoint,OneDrive -Sit "Credit Card Number" -MinCount 1 -WhatIf Create it (write, gated): add -Commit Enforce on a pilot subset: -ServiceSide -PolicyName "..." -Enforce -Locations "<site urls>" -Commit Return an enforced policy to simulation (rollback): -ServiceSide -PolicyName "..." -BackToSimulation -Commit Client-side recommend / auto-apply: -ClientSide -Label "..." -Sit "..." -MinCount 1 -Recommend (add -Commit) EXAMPLE .\IPAUTO-Configure.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -RuleName "AutoLabel Rule - Confidential - SPO+ODB" -Label "Confidential \ All Employees" -Workload SharePoint,OneDrive -Sit "Credit Card Number" -MinCount 1 -WhatIf .NOTES White-label Microsoft Purview tooling. No credentials stored. SIMULATION-first, -WhatIf default. Expected results: PASS = the intended New-/Set- change is printed (preview) or applied (-Commit) with no error; a service-side policy is created in a SIMULATION mode (TestWithoutNotifications), never Enable, until you explicitly -Enforce after sign-off. VERIFY IN YOUR TENANT: auto-labeling (client- and service-side) is E5/E5 Compliance and evolving; the exact Set-Label parameter surface for CLIENT-SIDE auto-labeling conditions is version-sensitive — this script previews the change and flags it; confirm the current parameter/behavior before -Commit. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$ClientSide, [switch]$ServiceSide,

How to run: PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. Preview a service-side SIMULATION policy (no change): .\IPAUTO-Configure.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" `-RuleName "AutoLabel Rule - Confidential - SPO+ODB" -Label "Confidential \ All Employees"` -Workload SharePoint,OneDrive -Sit "Credit Card Number" -MinCount 1 -WhatIf Create it (write, gated): add -Commit Enforce on a pilot subset: -ServiceSide -PolicyName "..." -Enforce -Locations "<site urls>" -Commit Return an enforced policy to simulation (rollback): -ServiceSide -PolicyName "..." -BackToSimulation -Commit Client-side recommend / auto-apply: -ClientSide -Label "..." -Sit "..." -MinCount 1 -Recommend (add -Commit) EXAMPLE .\IPAUTO-Configure.ps1 -ServiceSide -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -RuleName "AutoLabel Rule - Confidential - SPO+ODB" -Label "Confidential \ All Employees" -Workload SharePoint,OneDrive -Sit "Credit Card Number" -MinCount 1 -WhatIf .NOTES White-label Microsoft Purview tooling. No credentials stored. SIMULATION-first, -WhatIf default. Expected results: PASS = the intended New-/Set- change is printed (preview) or applied (-Commit) with no error; a service-side policy is created in a SIMULATION mode (TestWithoutNotifications), never Enable, until you explicitly -Enforce after sign-off. VERIFY IN YOUR TENANT: auto-labeling (client- and service-side) is E5/E5 Compliance and evolving; the exact Set-Label parameter surface for CLIENT-SIDE auto-labeling

conditions is version-sensitive — this script previews the change and flags it; confirm the current parameter/behavior before -Commit. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$ClientSide, [switch]$ServiceSide,

IPAUTO-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell and inventories the existing sensitivity LABELS (flagging which already carry client-side auto-labeling conditions), the LABEL POLICIES that publish them, and every SERVICE-SIDE auto-labeling policy with its MODE plus its rules — the baseline the auto-labeling build starts from. Confirms whether the target label exists and is published. Read-only — it changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module: Connect-IPPSSession .\IPAUTO-Discovery.ps1 -TargetLabel "Confidential \ All Employees" EXPECTED RESULTS AutoLabel-Posture-Estate-Discovery.csv (AutoLabel Posture - Estate - Discovery) listing labels, label policies, and service-side auto-labeling policies/rules with mode. PASS = the target label is present and published and the inventory completes without error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. VERIFY IN YOUR TENANT: automatic labeling (client- and service-side) is E5/E5 Compliance and evolving. This script reports what your session can read; it does not assert availability. > [CmdletBinding()] param([string]$TargetLabel = "Confidential \ All Employees", [string]$OutPath = ".\AutoLabel-Posture-Estate-Discovery.csv")

How to run: PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module: Connect-IPPSSession .\IPAUTO-Discovery.ps1 -TargetLabel "Confidential \ All Employees" EXPECTED RESULTS AutoLabel-Posture-Estate-Discovery.csv (AutoLabel Posture - Estate - Discovery) listing labels, label policies, and service-side auto-labeling policies/rules with mode. PASS = the target label is present and published and the inventory completes without error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. VERIFY IN YOUR TENANT: automatic labeling (client- and service-side) is E5/E5 Compliance and evolving. This script reports what your session can read; it does not assert availability. > [CmdletBinding()] param([string]$TargetLabel = "Confidential \ All Employees", [string]$OutPath = ".\AutoLabel-Posture-Estate-Discovery.csv")

IPAUTO-Report.ps1

Purpose: Exports a reporting snapshot for Microsoft Purview auto-labeling: every service-side auto-labeling policy with its MODE, applied label, and (where surfaced) its SIMULATION match/scan counts, plus its rule(s). Use it to review what a SIMULATION policy WOULD label (match counts) before enforcing, and to track coverage climbing after enforcement. Read-only — it writes only a CSV, changes no policy. HOW TO RUN PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. All policies: .\IPAUTO-Report.ps1 One policy: .\IPAUTO-Report.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" Optional label activity via Graph (read-only), if available: add -IncludeActivity EXAMPLE .\IPAUTO-Report.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -OutPath .\AutoLabel-Coverage-Report.csv .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Expected results: AutoLabel-Coverage-Report.csv (AutoLabel Coverage - Estate - Report) listing policies (mode, label, workload, simulation/match counts where exposed) and rules. PASS = the CSV is produced without error and the counts are reviewable with the data owner. VERIFY IN YOUR TENANT: simulation match/count property names and scan latency vary by tenant/version; a zero/blank count may mean "still scanning", not "nothing matches" — allow scan time before concluding. > [CmdletBinding()] param([string]$PolicyName, [string]$OutPath = ".\AutoLabel-Coverage-Report.csv", [switch]$IncludeActivity)

How to run: PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. All policies: .\IPAUTO-Report.ps1 One policy: .\IPAUTO-Report.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" Optional label activity via Graph (read-only), if available: add -IncludeActivity EXAMPLE .\IPAUTO-Report.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -OutPath .\AutoLabel-Coverage-Report.csv .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Expected results: AutoLabel-Coverage-Report.csv (AutoLabel Coverage - Estate - Report) listing policies (mode, label, workload, simulation/match counts where exposed) and rules. PASS = the CSV is produced without error and the counts are reviewable with the data owner. VERIFY IN YOUR TENANT: simulation match/count property names and scan latency vary by tenant/version; a zero/blank count may mean "still scanning", not "nothing matches" — allow scan time before concluding. > [CmdletBinding()] param([string]$PolicyName, [string]$OutPath = ".\AutoLabel-Coverage-Report.csv", [switch]$IncludeActivity)

IPAUTO-ScopeHealth.ps1

Purpose: Read-only health check for a Purview auto-labeling policy's SCOPE and readiness: which SharePoint sites / OneDrive accounts / Exchange (mailbox) locations the policy targets, whether it is scoped to a pilot subset or All, whether adaptive scopes are in use, and READINESS flags (label present + published, policy mode, cmdlet/role availability that stands in for licensing/service readiness). Helps confirm you enforce on the intended locations only. It NEVER writes. HOW TO RUN PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. .\IPAUTO-ScopeHealth.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" Export the scope map: add -OutPath .\AutoLabel-ScopeHealth.csv EXAMPLE .\IPAUTO-ScopeHealth.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -OutPath .\AutoLabel-ScopeHealth.csv .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Expected results: a per-location scope map + readiness flags, optionally to CSV (AutoLabel ScopeHealth - Estate - Report). PASS = the policy's locations resolve to the INTENDED scope (pilot subset, or All after expansion), with no unintended inclusions and readiness flags green. VERIFY IN YOUR TENANT: auto-labeling is E5/E5 Compliance and evolving; the LOCATION parameter names (SharePoint/OneDrive/Exchange, adaptive scope) vary by tenant/version — this reports what is exposed and flags anything it cannot resolve rather than asserting availability. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PolicyName, [string]$OutPath)

How to run: PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. .\IPAUTO-ScopeHealth.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" Export the scope map: add -OutPath .\AutoLabel-ScopeHealth.csv EXAMPLE .\IPAUTO-ScopeHealth.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -OutPath .\AutoLabel-ScopeHealth.csv .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Expected results: a per-location scope map + readiness flags, optionally to CSV (AutoLabel ScopeHealth - Estate - Report). PASS = the policy's locations resolve to the INTENDED scope (pilot subset, or All after expansion), with no unintended inclusions and readiness flags green. VERIFY IN YOUR TENANT: auto-labeling is E5/E5 Compliance and evolving; the LOCATION parameter names (SharePoint/OneDrive/Exchange, adaptive scope) vary by tenant/version — this reports what is exposed and flags anything it cannot resolve rather than asserting availability. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PolicyName, [string]$OutPath)

IPAUTO-Validate.ps1

Purpose: Read-only validation harness for a Purview auto-labeling policy. Confirms the service-side policy is in the EXPECTED state — in SIMULATION (TestWithoutNotifications/TestWithNotifications) before sign-off, or Enable only after — that its rule(s) exist and are enabled, and that a simulation scan/match set has been produced. Emits a single overall PASS/FAIL signal (exit code + summary) so it can gate the pipeline. It NEVER writes. HOW TO RUN PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. Validate a policy is still safely in simulation (pre-enforcement gate): .\IPAUTO-Validate.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -ExpectMode Simulation Validate a policy is enforcing after sign-off: .\IPAUTO-Validate.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -ExpectMode Enforce EXAMPLE .\IPAUTO-Validate.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -ExpectMode Simulation .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Expected results: PASS (exit 0) = policy mode matches -ExpectMode, at least one enabled rule exists, and (for simulation) a scan/match set is present for review. FAIL (exit 1) = a check did not hold. The PRECISION decision (are the matched items the right ones?) is the data owner's judgement on the match set — this script confirms the set EXISTS and the mode is correct, not that classification is right. VERIFY IN YOUR TENANT: auto-labeling is E5/E5 Compliance and evolving; simulation status property names and scan latency vary — treat "no match set yet" as allow-more-time, not necessarily a failure. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PolicyName, [ValidateSet('Simulation', 'Enforce')] [string]$ExpectMode = 'Simulation')

How to run: PowerShell 5.1+ or 7+ with the ExchangeOnlineManagement module, then Connect-IPPSSession. Validate a policy is still safely in simulation (pre-enforcement gate): .\IPAUTO-Validate.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -ExpectMode Simulation Validate a policy is enforcing after sign-off: .\IPAUTO-Validate.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -ExpectMode Enforce EXAMPLE .\IPAUTO-Validate.ps1 -PolicyName "AutoLabel Sim - Confidential - SPO+ODB" -ExpectMode Simulation .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Expected results: PASS (exit 0) = policy mode matches -ExpectMode, at least one enabled rule exists, and (for simulation) a scan/match set is present for review. FAIL (exit 1) = a check did not hold. The PRECISION decision (are the matched items the right ones?) is the data owner's judgement on the match set — this script confirms the set EXISTS and the mode is correct, not that classification is right. VERIFY IN YOUR TENANT: auto-labeling is E5/E5 Compliance and evolving; simulation status property names and scan latency vary — treat "no match set yet" as allow-more-time, not necessarily a failure. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PolicyName, [ValidateSet('Simulation', 'Enforce')] [string]$ExpectMode = 'Simulation')